

Harness Tri-Camada: memória como ambiente, API como tool, App como skill

Um runtime híbrido (ReAct + contexto-ambiente + skills) para o chat agêntico de marketing da AdzHub

Rafael Yran Azevedo

Candidato · Desafio Harness Agêntico · AdzHub Núcleo Fundacional, 2026

RESUMO. O gestor de marketing da AdzHub não precisa de um chatbot que responde: precisa de um agente que resolve tarefas compostas (cruzar gasto de anúncio com venda real, diagnosticar um CPA que subiu, montar a pauta de uma call) navegando o histórico da conta. Defendo que o erro comum, tratar tudo como *tool*, subutiliza o domínio. Proponho um harness híbrido de três mecanismos, um por camada: a memória (supercérebro: grafo + linha do tempo) é **hidratada como ambiente** antes do loop, não consultada às cegas; as APIs (Meta, CRM) são **tools** num loop ReAct; e os Apps de metodologia são **skills** encapsuladas que o harness invoca sem reimplementar. Um protótipo web de chat ilustra a tese sobre um dataset mock da Housewhey, com o *trace* do harness visível (hidratação → tools → observações → resposta) e cinco problemas de gestão plantados e não rotulados. Do protótipo sai também uma medição que muda decisão de projeto: num turno de cinco passos a entrada cresce de 2,9k para 11,3k tokens contra ~434 de saída, ou seja o custo de um harness ReAct é dominado pelo reenvio de observação, não pela geração. O que fica só no paper: a construção do grafo temporal real, permissões de escrita, avaliação automática e a camada de integrações (conectar contas de Meta, Google, Analytics e CRM), esta última deixada de fora por não conhecer o escopo completo pretendido, e não por descuido. O recorte consciente: nenhuma ação que escreve na conta; o MVP lê, diagnostica e propõe.

Palavras-chave: harness agêntico, ReAct híbrido, memória temporal, contexto como ambiente (RLM), skills, orquestração, marketing.

1. Introdução

O produto-alvo é um chat agêntico (no espírito do Cursor, mas para marketing): o gestor descreve uma tarefa e o agente orquestra contexto, Apps e APIs até uma ação útil. Antes de desenhar, separo dois conceitos que se confundem. O **modelo** é o LLM: prevê o próximo token e, no máximo, emite a intenção de chamar uma função. O **harness** é o runtime em volta: o loop que executa a chamada, injeta a observação de volta, guarda estado, impõe limites (allowlist de tools, teto de passos) e decide o que entra no contexto. Trocar de modelo troca o raciocínio; trocar de harness troca o que o agente consegue fazer com aquele raciocínio.

Uma imagem fixa a distinção. O LLM é um cérebro num pote: sabe muito, mas não vê o mundo nem age. O harness é o exoesqueleto que pluga esse cérebro na operação. O exoesqueleto restringe o foco (em vez de deixar o cérebro divagar, prende ele nos passos daquela tarefa), provê os músculos e as ferramentas (tools, memória, dados da conta) e força a saída num comportamento estrito. E faz isso *sem tocar no cérebro*: não mexemos nos pesos do modelo (isso seria treino), apenas embrulhamos software em volta e filtramos o que o LLM vê. Mesmo cérebro, exoesqueletos diferentes, agentes diferentes.

No início do estudo eu tratava harness como sinônimo de "loop de tool-calling": ligar o modelo a um punhado de funções e deixar rodar. Estudando os tipos de referência (ReAct [2], sandbox/CodeAct [3], sessão com permissões e skills, orquestração por grafo, contexto como ambiente [4]) e o SDK do OpenHands [1], o que mudou foi entender que a decisão de

arquitetura não é "qual loop", e sim **como cada recurso do domínio entra no runtime**: o que vira tool, o que vira memória e o que vira unidade de trabalho encapsulada. Um bom harness é, antes de tudo, uma política de contexto.

Tese. Para este domínio, um harness que trata suas três camadas com o mesmo mecanismo (tudo tool) é caro e frágil; defendo um híbrido em que *memória é ambiente, API é tool e App é skill*. As seções §2 a §7 definem o vocabulário, justificam a escolha contra as alternativas, mostram o que o protótipo ilustra e onde a solução quebra.

2. Preliminares

2.1. As peças do exoesqueleto

Retomo a imagem da introdução, porque ela dá nome a cada peça sem jargão. Um harness é o exoesqueleto que veste o cérebro, e tem seis partes. Descrevo cada uma pelo que ela faz no trabalho do gestor, não pelo termo técnico.

- **O ciclo de passos (loop).** O agente não responde de um fôlego só: ele dá voltas. É o mesmo movimento do gestor que abre o Gerenciador, vê que o CPA subiu, e só então decide abrir o CRM. Cada volta usa o que a anterior trouxe, e é isso que separa investigar de chutar.
- **As mãos (tools).** São as funções que o cérebro pode pedir para acionar: puxar os anúncios do Meta, os leads do CRM. O modelo não executa nada; ele pede, e o exoesqueleto executa.
- **O tato (observação).** O que a mão traz de volta. Sem isso o agente seria cego: é a observação que o faz mudar de ideia

no meio do caminho, quando o CRM mostra duas vendas onde o Meta prometia quatorze.

- **A memória.** O que ele já sabe antes de perguntar: quem é o cliente, quais campanhas existem, o que foi decidido na última call. Sem memória, toda pergunta começa do zero.
- **As travas (*allowlist* e *teto de passos*).** A lista fechada do que ele pode acionar e quantas voltas pode dar. É o que impede o exoesqueleto de sair andando sozinho, e o que separa um agente de um chatbot solto.
- **O que sobrevive entre as voltas (*estado*).** As mensagens, os resultados já obtidos, as decisões tomadas. Sem estado, a segunda volta esquece o que a primeira descobriu.

Nenhuma dessas peças está no modelo: todas são do runtime em volta dele. É por isso que dois produtos com o mesmo LLM entregam agentes diferentes.

2.2. O domínio AdzHub (como eu o leio)

A página do desafio expõe três camadas. **(i) Supercérebro:** a memória da operação, pessoas, cliente, campanhas e decisões ligadas em grafo (GraphRAG), com linha do tempo (contexto temporal). **(ii) Apps:** metodologias empacotadas (insight da semana, brief de criativo, diagnóstico, análise de criativos, mapa de solução). **(iii) APIs:** os dados da operação (Meta, Google, Analytics, CRM, WhatsApp). A observação central deste paper: essas três camadas têm *naturezas* diferentes, e o harness rende mais quando respeita essa diferença em vez de achatar tudo em "função que o LLM chama".

Por que o supercérebro é um grafo, e não uma tabela. O que o gestor precisa saber quase nunca está num registro só: está na *ligação* entre registros. "Por que o criativo de Creatina não subiu" se responde ligando criativo → tarefa de aprovação → pessoa que aprova → decisão registrada na call → restrição do mapa de solução. Numa tabela isso é um *join* que alguém precisa ter previsto; num grafo é caminhar de vizinho em vizinho, e é por isso que a recuperação pode partir de um nó qualquer citado na pergunta. As entidades que valem a pena existirem como nó são poucas e estáveis: cliente, campanha, criativo, pessoa, decisão, tarefa, restrição de marca. As arestas carregam o que a tabela perde: quem decidiu, para quem, a partir de qual conversa.

Por que a linha do tempo é uma camada e não um campo de data. Nenhuma afirmação de performance significa algo fora de uma linha de base: "o CPA subiu" só existe contra o CPA de antes. E, mais importante, fato de operação *vence*: "o cliente não quer falar em resultado garantido" era verdade em março e continua; "a verba é de R\$ 5.000" pode ter mudado na semana passada. Uma memória que só acumula texto responde com o fato revogado e soa confiante. Por isso trato tempo como parte da memória (o que valia quando), no espírito do grafo temporal com invalidação [5], e não como uma coluna `created_at` que ninguém consulta.

Por que um App não é só mais uma tool. As duas são "função que o harness chama", e é aí que a distinção parece

pedante. A diferença é o que cada uma carrega: a tool carrega *dado* (o gasto do anúncio é o que é), e o App carrega *opinião do negócio* (o que conta como criativo saturado, quando recomendar pausar, o que a marca não pode falar). Colocar metodologia no prompt tem três custos que a versão encapsulada não paga: ela é reprocessada pelo LLM a cada turno (token), pode ser reinterpretada de um jeito diferente a cada turno (consistência), e deixa de ser versionável pelo time que a escreveu (produto). Um App é a metodologia da AdzHub virando código chamável; se ela vira parágrafo de prompt, ela vira sugestão.

O que os canais mudam. WhatsApp e reuniões não são mais uma fonte de métrica, são onde mora o *porquê*: a aprovação que travou, o combinado da call, a reclamação que antecede o pedido de relatório. É a única camada que explica decisão humana, e é por isso que ela entra como memória e não como API de dado. Já Analytics ao lado de Meta e CRM cria um problema que o harness precisa assumir em vez de esconder: **três sistemas contam a mesma venda de três jeitos** (janelas de atribuição e modelos diferentes). O desenho aqui não elege um vencedor em silêncio; expõe a divergência, que é exatamente o quinto problema plantado no dataset (§4.2).

2.3. Cinco famílias de harness (o terreno estudado)

Antes de escolher, mapeei as famílias que aparecem na literatura e nos produtos. Resumo o que cada uma propõe de fato, não o rótulo, porque a diferença entre elas não é de estilo: é de *onde* o trabalho acontece.

ReAct [2] intercala raciocínio e ação no mesmo traço: o modelo escreve o que pensa, escolhe a ação, lê a observação e revisa o pensamento. O ganho não é a chamada de função em si (qualquer loop faz isso), é a observação poder *corrigir* o raciocínio no meio do caminho. **CodeAct** [3] muda o espaço de ação: em vez de JSON, o agente escreve código executável, o que dá laço, condicional e reaproveitamento de resultado anterior de graça, ao custo de um sandbox. O **SDK do OpenHands** [1] mostra o harness como produto de engenharia: fluxo de eventos como estado, runtime isolado, tools plugáveis e uma camada que inspeciona a ação antes de executar. **RLM** [4] muda a pergunta: em vez de empurrar tudo para dentro do prompt, trata o contexto como um ambiente que o modelo percorre. A nota da Anthropic [6] separa *workflow* (caminho predefinido em código) de *agente* (o modelo dirige o próprio processo) e recomenda a coisa mais simples que resolve, o que é um argumento direto contra orquestrar este chat por grafo de estados. E a memória por grafo temporal [5] fecha o mapa pelo lado do dado: fatos com validade, que mudam e são invalidados no tempo.

A Tabela 1 põe as cinco lado a lado com o veredito que cada uma recebeu neste desenho. A coluna que decide não é "onde brilha", é **onde falha no chat do gestor**: é ela que explica por que a resposta não é escolher uma, e sim compor.

Tabela 1. As cinco famílias estudadas e o veredito de cada uma para o chat agêntico de marketing.

Família	Onde o trabalho acontece	Brilha quando	Onde falha no chat do gestor	Veredito aqui
ReAct [2]	loop pensa → age → observa	a tarefa é aberta e exige investigar	sozinho, memória vira "mais uma tool" e o modelo precisa lembrar de buscar contexto	adotado como núcleo
CodeAct / sandbox [3]	código executado em ambiente isolado	cálculo encadeado, engenharia, transformação de dado	soma execução e latência para um trabalho que é cruzar tabela e aplicar método	fora do MVP; volta como tool de verificação (§7)
Sessão com permissões e skills [1]	unidade de trabalho encapsulada + porteiro antes da ação	metodologia repetível e ação que escreve	permissão resolve um risco que o MVP não corre, porque ele é só-leitura	metade: skills sim, permissões adiadas
Orquestração por grafo de estados	caminho desenhado antes de rodar	o fluxo é conhecido e se repete igual	é uma tarefa diferente por mensagem; caminho fixo engessa e falha no caso novo [6]	fora (o grafo aqui é de dados, não de controle)
Contexto como ambiente (RLM) [4]	recuperação antes do loop	todo pedido pressupõe histórico da conta	sozinho não age: não chama API nem executa metodologia	adotado para a camada de memória

3. Arquitetura

3.1. A escolha: um híbrido de três mecanismos

A Tabela 1 já traz o veredito de cada família. O que explico aqui é a *composição*, porque é nela que está a tese: a decisão não é "qual das cinco", é **qual mecanismo para qual camada**. Nenhuma delas cobre as três sozinha.

Núcleo ReAct, porque a tarefa é aberta. O gestor não manda um comando por vez, ele descreve um problema ("o CPA subiu, o que houve"), e o caminho até a resposta só se descobre andando: o número do Meta é que faz querer abrir o CRM. Isso é exatamente o que o loop pensa-age-observa entrega, e é o motivo de eu não usar um grafo de controle: caminho desenhado antes só funciona quando o fluxo se repete, e aqui cada mensagem é uma tarefa diferente [6].

Memória fora do loop, porque toda pergunta pressupõe histórico. Esta é a correção que o ReAct puro precisa neste domínio. Se o supercérebro for só mais uma tool, o modelo tem que *lembrar* de consultá-lo, e erra dos dois lados: às vezes responde no vácuo, às vezes gasta dois passos buscando o que já devia estar na mão. Trato a memória como ambiente [4]: antes do loop, o harness recupera um pacote (nós relevantes do grafo + eventos recentes da timeline) e injeta no prompt. O agente já começa situado, e a memória continua disponível como tool para o aprofundamento sob demanda.

Apps como skills, porque metodologia é produto e não prompt. A parte de *skills* da referência de sessão eu adoto inteira: o App é chamado, devolve ranking e recomendação, e o harness usa a saída sem recriar o método. A parte de *permissões* eu adio de propósito, porque o MVP não escreve na conta (§3.3). Uso metade da referência, conscientemente, e digo qual metade.

O híbrido, então: **RLM para a memória, ReAct para as APIs, skills para os Apps**. A tese não está em nenhuma das três escolhas isoladas, está na composição, e o que a justifica é a Tabela 1: cada família falha justamente na camada que outra cobre.

3.2. O que é tool, o que é memória, o que é app

Memória (supercérebro), hidratada como ambiente. Antes do loop, o harness chama uma recuperação (`search_client_context + get_timeline`) semeada pela intenção e injeta um pacote compacto no *system prompt*. Durante o loop, o modelo ainda pode aprofundar (buscar uma ata com `search_conversations`). Memória é ambiente por padrão, tool sob demanda.

API (Meta, CRM), tools clássicas. Sem estado, chamadas dentro do loop: `list_ads` e `get_ad_insights` (gasto, série semanal, saturação), `get_leads` (venda real). O join Meta×CRM por `utm_content = ad_id` é trabalho do *agente*, não da tool: é aí que mora a inteligência de "não confiar só no gerenciador".

App (metodologia), skills encapsuladas. `run_app_analise_criativos` devolve ranking + recomendação (seguir/pausar/variá); `get_mapa_solucão` devolve a ficha da marca (o que não pode falar). O harness chama o app e usa a saída; não recria a metodologia no prompt. Isso mantém a metodologia versionável no produto e barata em tokens.

3.3. Trade-offs aceitos de propósito

- **Fidelidade vs simplicidade:** a hidratação é uma recuperação simples (match no grafo + últimos N eventos), não um recuperador semântico com embeddings. Aceito recall menor por latência baixa e previsibilidade; num MVP, contexto errado é pior que contexto raso.
- **Latência vs profundidade:** `max_steps = 8` e allowlist fechada. Prefiro cortar cedo e responder com o que há a divagar.
- **Custo:** Apps como skills e `get_leads` com sumário cortam tokens; a metodologia não é reprocessada pelo LLM a cada turno.
- **Segurança:** só-leitura. Nenhuma tool escreve na conta (não pausa anúncio, não mexe em verba). O agente propõe; o humano executa. Remove a classe inteira de risco de ação indevida e é coerente com "o gestor decide".

- **Simplicidade do MVP:** um cliente, tools que leem mocks. Sem multi-tenant, sem auth, sem escrita.

4. Sistema

4.1. O que o protótipo ilustra

Um chat web roda o harness sobre um dataset mock da Housewhey, com um agente de persona própria (**NEXO**, estrategista de performance). Cada tarefa do gestor abre uma **conversa separada**, com histórico e trace próprios: diagnosticar um CPA e montar a pauta da call são assuntos diferentes, e juntá-los num fio só faria cada passo do loop reenviar contexto que não é daquela tarefa, que é exatamente o custo medido no Quadro 1. O turno pertence à conversa e não à tela: trocar de fio no meio de uma resposta é permitido, e a resposta continua chegando no fio que a pediu, porque o estado do turno é da conversa. A persona vive num arquivo separado do *runtime*, de propósito: o loop é mecanismo, a persona é dado do harness, e o mesmo runtime serve outra persona sem alteração de código. Cada turno mostra, num painel de *trace*, as fases do harness: hidratação do supercérebro (fase), depois as tool calls (com *badge* por camada: memória/api/app), com args e observação inspecionáveis, e a resposta ancorada nos números. Dois motores compartilham o mesmo trace: **(a) Simulado**, sem chave, um planejador roteirizado que executa

as tools reais e compõe a resposta a partir do dado (prova que o dado sustenta a conclusão); **(b) LLM**, via OpenRouter, o loop ReAct de verdade decide as chamadas.

Um turno concreto (intent → memória/tools → observação → resposta), para "o CPA do Ômega 3 subiu, diagnostique":

- **Hidratação:** nós do grafo (campanha Ômega 3, time, tarefa de aprovação) + timeline recente entram no prompt.
- **Loop:** `list_ads` (gasto por anúncio) → `get_leads` (venda real) → `get_ad_insights` (hook_rate 32%→18%, frequência 1,8→4,8) → `run_app_analise_criativos` (recomenda pausar).
- **Observações combinadas:** o criativo de depoimento tem CPA R\$ 2.100 contra R\$ 100 do antes/depois, saturou e sozinho estourou o orçamento (R\$ 6.200 vs R\$ 5.000).
- **Resposta:** causa raiz + próximo passo: pausar o depoimento resolve CPA, fadiga e verba de uma vez.

Observabilidade inclui custo, e o custo tem um formato surpreendente. Um turno de agente não é uma chamada de LLM, são N: uma por passo do loop, e *cada passo reenvia a conversa inteira mais todas as observações já colhidas*. Isso é estrutural em qualquer harness ReAct, e é invisível para quem só olha o preço por token. O protótipo mostra o consumo por chamada, por turno e por sessão, e o Quadro 1 traz o turno de diagnóstico acima medido passo a passo.

QUADRO 1. Custo real de um turno de 5 passos (diagnóstico do Ômega 3). A entrada cresce quase 4x ao longo do turno; a saída do turno inteiro cabe em ~434 tokens.

Passo	O que o modelo decidiu ali	Entrada (tokens)	Por que cresceu
1	chamar <code>list_ads</code>	2.900	prompt + memória hidratada
2	chamar <code>get_leads</code>	3.800	+ observação do passo 1
3	chamar <code>get_ad_insights</code>	10.400	+ leads (a observação mais pesada)
4	chamar <code>run_app_analise_criativos</code>	10.700	+ série semanal
5	escrever a resposta	11.300	+ recomendação do App

A leitura: o custo de um harness ReAct é dominado pelo *reenvio* de observação, não pela geração. Uma tool que devolve JSON cru não custa uma vez, custa uma vez por passo restante do turno. É esse número, e não o gosto do desenvolvedor, que decide se `get_leads` devolve a lista inteira ou um sumário. (Valores da estimativa do modo simulado, que reconstrói o loop passo a passo; no modo LLM o mesmo painel mostra o *usage* medido pela API, e medido nunca aparece sem rótulo ao lado de estimado.)

4.2. Datasets: o que é fake, o que dá para testar

Sete arquivos JSON (um cliente, cruzados: `ad_id` = `utm_content`, pessoas no grafo, timeline e WhatsApp). Cada arquivo é uma tool (Tabela 2). Tudo é fake, mas coerente ponta a ponta; **cinco problemas de gestão estão plantados e não**

rotulados: CPA estourado, criativo saturado, origem declarada que mente contra o UTM, aprovação travada por *claim* proibido, e spend vs budget. O avaliador cola três prompts (relatório, diagnóstico, origem dos leads) e vê o harness orquestrar duas ou mais tools e chegar à conclusão sem que ela esteja escrita no dado.

Tabela 2. Peças do harness: o que é cada uma e onde vive.

Peça (tool)	Camada	Papel na tese	Estado no protótipo
<code>search_client_context</code>	Supercérebro · grafo	Memória (ambiente): quem/o quê da conta	simulado sobre mock; visível no trace
<code>get_timeline</code>	Supercérebro · temporal	Memória (ambiente): histórico recente	simulado sobre mock; visível no trace
<code>search_conversations</code>	Memória de canal	Memória (sob demanda): o porquê (aprovação)	simulado sobre mock; visível no trace

list_ads / get_ad_insights	API · Meta Ads	Tool: gasto, série, saturação	simulado sobre mock; visível no trace
get_leads	API · CRM	Tool: venda real, atribuição	simulado sobre mock; visível no trace
run_app_analise_criativos	App · metodologia	Skill: ranking + recomendação	simulado sobre mock; visível no trace
get_mapa_solucao	App · marca	Skill: restrições ("não pode falar")	simulado sobre mock; visível no trace

4.3. OpenRouter e troca de modelo

No painel `Configurar`, o avaliador escolhe o motor (Simulado ou LLM) e cola a `OPENROUTER_API_KEY`, guardada só em `sessionStorage` e enviada apenas à OpenRouter, nunca a um servidor nosso. O modelo sai de um dropdown montado a partir da API pública da OpenRouter, buscada pelo browser do avaliador e filtrada por quem suporta *tool-calling* (sem isso o harness não existe: o modelo responderia texto sem nunca tocar numa tool). A lista vem em **três grupos**, porque duas perguntas diferentes precisam de resposta: *recomendados*, ranqueados do mais forte (#1) ao mais fraco e com um por família, responde "qual eu uso"; *grátis* reúne os de custo zero, para avaliar sem gastar; e o **catálogo inteiro**, em ordem alfabética, responde "quero este aqui". Uma quarta opção aceita um id digitado, para o modelo que nasceu depois desta lista. A lista é uma `listbox` própria, com filtro por id: o `<select>` nativo abre sempre rolado até o item escolhido, o que com centenas de modelos jogava os recomendados para fora da tela, e essa rolagem não é configurável. Fixar ids à mão envelhece e vira 400 na hora da avaliação; o ranking é heurística declarada (família, preço de saída, contexto) casada por padrão de nome, e sem rede cai numa lista local, que não anuncia nada como grátis porque preço ausente não é preço zero. O modo Simulado dispensa chave e serve quem só quer ver o harness orquestrar.

5. Notas de execução

PDF anexo (este). Demo opcional: chat web estático, roda de `file://` por duplo clique ou de qualquer host, sem backend. LLM via OpenRouter, chave só na sessão do browser. Dados mock gerados de forma determinística a partir do

`dataset_prompt`. Consumo de tokens medido pelo `usage` da própria API no modo LLM e estimado (sempre marcado com $\approx$) no simulado, onde não há LLM para medir. A resposta é streamada (SSE), com duas notas de engenharia que só aparecem ao streamar um loop *com tools*: os `tool_calls` chegam **fatiados** (nome e argumentos em pedaços indexados) e precisam ser remontados no transporte, senão o nome da tool chega quebrado; e o texto escrito *antes* de uma tool é raciocínio, não resposta, então migra para o trace em vez de ficar colado na resposta final. Comandos de sessão (`/tools`, `/uso`) rodam localmente, fora do harness. A entrada aceita **arquivo** (CSV, TSV, JSON, TXT, MD) lido no browser e injetado no contexto como bloco marcado, cortado em 40 mil caracteres pelo próprio harness, porque num loop ReAct o anexo não custa uma vez, custa uma vez por passo restante do turno. O **ditado** usa a API de fala do navegador, e não Whisper, por uma razão de escopo: a OpenRouter expõe apenas `/chat/completions`, então transcrever exigiria uma segunda chave de outro provedor na mesma UI, o que dobra a superfície de exposição de credencial num desafio que define a chave como sendo a da OpenRouter. O que é fake: todas as métricas e nomes. O que "parece real": o cruzamento e o trace, porque as tools leem o mesmo dado que a resposta cita, e o avaliador pode abrir cada passo.

6. Pontos críticos por tarefa do gestor

Uma arquitetura só se avalia contra o trabalho real. Percorro as tarefas do dia a dia do gestor da AdzHub (as que os Apps cobrem, mais as duas que atravessam tudo, relatório e call) e digo, em cada uma, onde *este* desenho ajuda e onde ele quebra. A Tabela 3 é o resumo; os dois casos mais graves ganham parágrafo depois.

Tabela 3. Onde o Harness Tri-Camada ajuda e onde ele quebra, por tarefa do gestor.

Tarefa	O que o harness faz	Ponto crítico	O que no desenho mitiga (e o que não)
Insight da semana (relatório)	cruza gasto do Meta com venda do CRM por <code>utm_content</code> e ordena por custo por venda	o cruzamento vale o que vale o UTM; atribuição divergente entre Meta e CRM muda o ranking	expõe a divergência em vez de escolher um lado; não reconcilia janelas de atribuição
Diagnóstico (o CPA subiu)	hidrata a campanha, puxa série e frequência, chama o App de criativos	o agente correlaciona, não prova causa; hidratação rasa pode não trazer o nó certo	separa fato de hipótese na resposta; não faz teste estatístico
Análise de criativos	invoca o App e usa ranking + recomendação (seguir/pausar/variante)	o App julga pelo número, sem ver a peça: criativo bom com oferta errada é indistinguível	metodologia versionada fora do prompt; não analisa imagem nem vídeo
Brief de criativo	gera o brief a partir do que venceu, com o mapa de solução no contexto	é a única tarefa <i>geradora</i> : o LLM pode propor copy que a marca não pode falar	injeta <code>get_mapa_solucao</code> ; não garante, porque restrição no prompt é instrução, não trava
Mapa de solução	lê a ficha da marca como skill e a usa como limite do que sugerir	a ficha é um retrato: cliente muda de posicionamento e ninguém reescreve o	tempo é camada de memória (fato vence); não detecta sozinho que a ficha envelheceu

(restrições)		documento	
Pauta de call	lê timeline e conversas, monta o que mudou e o que ficou pendente	o harness não sabe o que não foi registrado: memória esparsa vira pauta genérica	puxa o canal (WhatsApp) junto; não inventa o que ninguém anotou
Responder o cliente	recupera o histórico do canal e propõe a resposta	é onde uma ação errada custa relacionamento, não só verba	só-leitura por desenho: o agente redige, o humano envia; não automatiza o envio

O crítico do relatório é atribuição, não join. No mock, `utm_content = ad_id` sempre bate, e é isso que faz a demo parecer limpa. Na conta real, UTM falta, vem errado, e o mesmo lead toca três anúncios antes de comprar; Meta credita por visualização dentro da janela dele, o CRM credita o último clique, e os dois estão "certos" pelas próprias regras. Um agente que escolhe uma fonte e segue em frente produz um número que ninguém consegue auditar. Por isso o quinto problema plantado no dataset é exatamente esse (leads que declaram Google mas carregam UTM do Meta), e por isso a persona é instruída a *expor* a discrepância. Resolver de verdade é um projeto de atribuição, não um prompt.

O crítico do diagnóstico é confundir correlação com causa. "O criativo saturou" é a leitura mais provável quando frequência sobe e hook rate cai, mas continua sendo leitura: pode ser sazonalidade, concorrente novo no leilão, ou mudança na página de destino que o agente nem enxerga. O desenho responde a isso separando na saída o que é fato medido, o que é hipótese e o que é recomendação, o que devolve ao gestor a decisão de confiar. O que ele não faz é medir a confiança dessa hipótese, e isso não é detalhe: um agente que fala com a mesma segurança nos dois casos treina o gestor a parar de conferir.

7. Limitações e próximos passos

Além dos pontos críticos por tarefa: só-leitura por escolha (não fecho o loop de execução); um cliente só (sem multi-tenant nem auth); memória mock, sendo que o grafo temporal de verdade (resolução de entidade, invalidação, decaimento) é o trabalho pesado que fica de fora. O protótipo também não trata sessão longa: hoje o turno é curto, e o Quadro 1 mostra por que isso não escala sozinho, mas a política de compactação de contexto ao longo de uma conversa de quarenta mensagens não está desenhada.

O que ficou de fora de propósito, e por quê. Não existe no protótipo uma **camada de integrações**: nem tela de conectar contas, nem OAuth por cliente, nem cofre de credenciais, nem estado de conexão (token expirado, permissão revogada, conta trocada), nem seleção de conta em quem tem várias. As tools leem o mock; num produto elas leriam Meta, Google Ads, Analytics, CRM e WhatsApp com credencial de cada cliente. Também não há **geração de criativo** (imagem ou vídeo, via Gemini/Imagen ou equivalente): o agente escreve o *brief*, não produz a peça.

A razão tem duas metades, e a segunda é a mais honesta. A primeira é recorte: isto é um protótipo para defender uma tese

de harness, e conexão de conta é trabalho de produto, não de runtime. A segunda é que **eu não conheço o escopo completo do harness que a AdzHub pretende**: quem conecta a conta (a agência ou o cliente), como as credenciais são guardadas e rotacionadas, como funciona multi-conta e multi-cliente, que permissões cada perfil tem, o que acontece quando o token cai no meio de um turno. Desenhar uma superfície de credencial sem essas respostas seria inventar requisito, e requisito inventado em cima de credencial de anúncio é o tipo de erro que só aparece depois, caro. Preferi deixar o buraco visível a preencher com suposição.

O que dá para afirmar é que a arquitetura acomoda isso sem mexer no loop: pela tese das três camadas, **uma integração nova é uma tool nova na allowlist** (o loop ReAct não muda), a área de conectar contas é produto em volta do harness, e a geração de criativo entraria como *App* (skill), não como tool crua, porque ela carrega metodologia e restrição de marca. O mesmo vale para a hidratação: mais fontes significam mais nós no grafo, não outro mecanismo.

Com mais uma semana eu construiria: (a) recuperação de memória de verdade, um grafo temporal com embeddings e decaimento (no espírito de Graphiti/Zep [5]), para a hidratação parar de depender de match por substring; (b) uma tool isolada de verificação numérica, o único lugar onde um mini-sandbox se paga, para o join Meta×CRM ficar auditável; (c) confirmação de escrita (pausar anúncio) com o humano no meio. **O que eu deliberadamente não construiria**: um grafo de controle por cima do chat (mata a abertura da tarefa), execução de código arbitrário no MVP (risco sem retorno) e memória de longo prazo que aprende sozinha sem revisão, porque num domínio com o dinheiro do cliente, contexto errado propaga erro.

Referências

1. OpenHands. *The OpenHands Software Agent SDK*. arXiv:2511.03690, 2026.

2. S. Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629, 2023.

3. X. Wang et al. *Executable Code Actions Elicit Better LLM Agents (CodeAct)*. arXiv:2402.01030, 2024.

4. *Recursive Language Models*. arXiv:2512.24601, 2026.

5. P. Rasmussen et al. *Zep / Graphiti: memória de agente por grafo de conhecimento temporal*. 2025.

6. Anthropic. *Building Effective Agents*. Nota de engenharia, 2024.